IMPACT
College of Engineering & Applied Sciences
ICEAS

LAB MANUAL

AY: 2025 – 26 (ODD SEMESTER)
SEMESTER: 7
DEPT(s): AI & ML
SUBJECT NAME: MACHINE LEARNING II
SUBJECT CODE: BAI702

Syllabus – Machine Learning II

Machine Learning II		Semester	7
Course Code	BAI702	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	3:0:2:0	SEE Marks	50
Total Hours of Pedagogy	40 hours Theory + 8-10 Lab slots	Total Marks	100
Credits	04	Exam Hours	3
Examination nature (SEE)	Theory/practical		

Course objectives:

- To introduce concept learning and General to specific learning
- To learn set of rules using Sequential Covering approach
- To make decisions on decision by committee
- To understand similarities using unsupervised learning .
- To understand Markov Chain Monte Carlo (MCMC) and Graphical Methods

Teaching-Learning Process (General Instructions)
These are sample Strategies; that teachers can use to accelerate the attainment of the various course outcomes.

1. Lecturer method (L) need not to be only a traditional lecture method, but alternative effective teaching methods could be adopted to attain the outcomes.
2. Use of Video/Animation to explain functioning of various concepts.
3. Encourage collaborative (Group Learning) Learning in the class.
4. Ask at least three HOT (Higher order Thinking) questions in the class, which promotes critical thinking.
5. Adopt Problem Based Learning (PBL), which fosters students' Analytical skills, develop design thinking skills such as the ability to design, evaluate, generalize, and analyze information rather than simply recall it.
6. Introduce Topics in manifold representations.
7. Show the different ways to solve the same problem with different logic and encourage the students to come up with their own creative ways to solve them.
8. Discuss how every concept can be applied to the real world - and when that's possible, it helps improve the students' understanding

MODULE-1

Introduction: Well-Posed Learning Problems, Designing a Learning System, Perspectives and Issues in Machine Learning.
Concept Learning and the General-to-Specific Ordering: A Concept Learning Task, Concept Learning as Search, Find-S: Finding a Maximally Specific Hypothesis, Version Spaces and the Candidate-Elimination Algorithm, Remarks on Version Spaces and Candidate-Elimination, Inductive Bias.
Text Book 1 : Ch 1 & 2

MODULE-2

Learning Sets of Rules: Sequential Covering Algorithms, Learning Rule Sets: Example-Based Methods, Learning First-Order Rules, FOIL: A First-Order Inductive Learner.
Analytical Learning: Perfect Domain Theories: Explanation-Based Learning, Explanation-Based Learning of Search Control Knowledge, Inductive-Analytical Approaches to Learning.
Text Book 1 : Ch 10 & 11

MODULE-3

Decision by Committee: Ensemble Learning: Boosting: Adaboost , Stumping, Bagging: Subagging, Random Forests, Comparison With Boosting, Different Ways To Combine Classifiers.
Unsupervised Learning: The K-MEANS algorithm : Dealing with Noise ,The k-Means Neural Network , Normalisation ,A Better Weight Update Rule ,Using Competitive Learning for Clustering.
Text Book 2: Chap 13 and 14.1

MODULE-4

Unsupervised Learning: Vector Quantisation, the self-organising feature map , The SOM Algorithm, Neighbourhood Connections, Self-Organisation, Network Dimensionality and Boundary Conditions, Examples of Using the SOM. **Markov Chain Monte Carlo (MCMC) Methods:** Sampling : Random Numbers ,Gaussian Random Numbers ,Monte Carlo Or Bust ,The Proposal Distribution , Markov Chain Monte Carlo. **Text Book 2: Chap 14.2, 14.3, 15**
MODULE-5
Graphical Models: Bayesian Networks : Approximate Inference , Making Bayesian Networks , Markov Random Fields , Hidden Markov Models (Hmms), The Forward Algorithm , The Viterbi Algorithm , The Baum–Welch Or Forward–Backward Algorithm , Tracking Methods , The Kalman Filter, The Particle Filter. **Text Book 2 : Chap 16**

PRACTICAL COMPONENT OF IPCC *(May cover all / major modules)*

Sl.NO	Experiments
1	Read a dataset from the user and i. Use the Find-S algorithm to find the most specific hypothesis that is consistent with the positive examples. Ii. What is the final hypothesis after processing all the positive examples? Using the same dataset, apply the Candidate Elimination algorithm. Determine the final version space after processing all examples (both positive and negative). What are the most specific and most general hypotheses in the version space?
2	Read a dataset and use an **example-based method** (such as **RIPPER** or **CN2**) to generate a set of classification rules . Apply the **FOIL algorithm** (First-Order Inductive Learner) to learn first-order rules for predicting.
3	Read a supervised dataset and use bagging and boosting technique to classify the dataset. Indicate the performance of the model.
4	Read an unsupervised dataset and group the dataset based on similarity based on k-means clustering .
5	Read a dataset and perform unsupervised learning using SOM algorithm.
6	Write a function to generate **uniform random numbers** in the interval [0, 1]. Use this function to generate 10 random samples and evaluate f(x) for each sample. What are the sampled function values? Using the samples generated in the previous step, estimate the integral I using the **Monte Carlo method.**
7	Read a dataset and indicate the likelihood of an event occurring using Bayesian Networks.
8	Refer to the dataset in question 7 and indicate inferences based on the sequence of steps .

Course outcomes (Course Skill Set):
At the end of the course, the student will be able to:

1. Apply concept learning and General to specific learning
2. Design models to classify supervised data .
3. To analyze methods to identify similarities using unsupervised learning .
4. To understand Markov Chain Monte Carlo (MCMC) and Graphical Methods.

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

1.Read a dataset from the user and

1. Use the Find-S algorithm to find the most specific hypothesis that is consistent with the positive examples.
2. ii. What is the final hypothesis after processing all the positive examples? Using the same dataset, apply the Candidate Elimination algorithm.

Determine the final version space after processing all examples (both positive and negative). What are the most specific and most general hypotheses in the version space?

```
import pandas as pd

# ------------------ Read dataset -------------------
def read_dataset():
    file_path = input("Enter path to CSV file: ")
    df = pd.read_csv(file_path)
    X = df.iloc[:, :-1].values
    y = df.iloc[:, -1].values
    return X, y

# ------------------ Find-S Algorithm -------------------
def find_s_algorithm(X, y):
    hypothesis = ['φ'] * X.shape[1]
    for i, instance in enumerate(X):
        if y[i].lower() == 'yes':
            if hypothesis == ['φ'] * X.shape[1]:
                hypothesis = list(instance)
            else:
                for j in range(len(hypothesis)):
                    if hypothesis[j] != instance[j]:
                        hypothesis[j] = '?'
    return hypothesis

# ------------------ Candidate Elimination -------------------
def is_consistent(hypothesis, instance):
    return all(h == '?' or h == x for h, x in zip(hypothesis, instance))
```

```
def candidate_elimination(X, y):
    num_attributes = X.shape[1]
    # Initial boundaries
    G = [['?' for _ in range(num_attributes)]]
    S = [['φ' for _ in range(num_attributes)]]

    domains = [list(set(X[:, i])) for i in range(num_attributes)]  # domain of each attribute

    for i, instance in enumerate(X):
        if y[i].lower() == 'yes':
            # Remove from G any hypothesis inconsistent with the positive example
            G = [g for g in G if is_consistent(g, instance)]

            # Generalize S if needed
            for s in S:
                for j in range(num_attributes):
                    if s[j] == 'φ':
                        s[j] = instance[j]
                    elif s[j] != instance[j]:
                        s[j] = '?'
        else:
            # Remove from S any hypothesis that is consistent with negative example
            S = [s for s in S if not is_consistent(s, instance)]

            # Specialize G to exclude the negative instance
            new_G = []
            for g in G:
                if is_consistent(g, instance):
                    for attr_index in range(num_attributes):
                        if g[attr_index] == '?':
                            for val in domains[attr_index]:
                                if val != instance[attr_index]:
                                    new_hypothesis = g.copy()
                                    new_hypothesis[attr_index] = val
                                    # Keep only if consistent with all positive examples
                                    if all(is_consistent(new_hypothesis, x) for k, x in enumerate(X) if y[k].lower() == 'yes'):
                                        new_G.append(new_hypothesis)
            G = new_G

            # Remove duplicates from G
            final_G = []
            for g in G:
                if g not in final_G:
                    final_G.append(g)

            return S, final_G

        # ------------------- Main --------------------
        if __name__ == "__main__":
            X, y = read_dataset()

            print("\n--- Find-S Algorithm ---")
            final_hypothesis = find_s_algorithm(X, y)
            print("Final Most Specific Hypothesis (Find-S):", final_hypothesis)

            print("\n--- Candidate Elimination Algorithm ---")
            S, G = candidate_elimination(X, y)
            print("S (Most Specific Hypotheses):", S)
            print("G (Most General Hypotheses):", G)
```

OUTPUT

```
Enter path to CSV file:  C:\Users\sheet\OneDrive\Documents\ICEAS College\2025 Odd Sem\ML2 Lab\Pgm1\Weather.csv

--- Find-S Algorithm ---
Final Most Specific Hypothesis (Find-S): ['Sunny', 'Warm', '?', 'Strong', '?', '?']

--- Candidate Elimination Algorithm ---
S (Most Specific Hypotheses): [['Sunny', 'Warm', '?', 'Strong', '?', '?']]
G (Most General Hypotheses): [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]
```

2. Read a dataset and use an example-based method (such as RIPPER or CN2) to generate a set of classification rules . Apply the FOIL algorithm (First-Order Inductive Learner) to learn first-order rules for predicting

```
import pandas as pd
import math

# --------------------------------
# 1. Load Dataset from CSV
# --------------------------------
df = pd.read_csv("C:\\Users\\sheet\\OneDrive\\Documents\\ICEAS College\\2025 Odd Sem\\ML2 Lab\\Pgm2\\Weather1.csv")

print("Dataset Loaded:", df.shape)
print(df.head())  # preview first few rows

# --------------------------------
# 2. Example-based Rules (like CN2 style)
# --------------------------------
def example_based_rules(df, target="Play"):
    rules = []
    for attr in df.columns:
        if attr == target:
            continue
        for val in df[attr].unique():
            subset = df[df[attr] == val]
            if not subset.empty and len(subset[target].unique()) == 1:
                # condition perfectly predicts the class
                rules.append((attr, val, subset[target].iloc[0]))
    return rules

# --------------------------------
# 3. FOIL Gain
# --------------------------------
def foil_gain(p_new, n_new, p_old, n_old):
    """FOIL information gain"""
    if p_new == 0:
        return -1
    return p_new * (math.log((p_new/(p_new+n_new)) + 1e-9) - math.log((p_old/(p_old+n_old)) + 1e-9))
# --------------------------------
# 4. FOIL Algorithm with Stopping & Pruning
# --------------------------------
def foil(df, target_class, target="Play"):
    rules = []
    pos = df[df[target] == target_class]
    neg = df[df[target] != target_class]

    while not pos.empty:
        rule = []
        covered_pos, covered_neg = pos.copy(), neg.copy()

        # Build a rule
        while not covered_neg.empty:
            best_gain, best_literal, best_pos, best_neg = -1, None, None, None

            for attr in df.columns:
                if attr == target:
                    continue
                for val in df[attr].unique():
                    pos_cov = covered_pos[covered_pos[attr] == val]
                    neg_cov = covered_neg[covered_neg[attr] == val]

                    # If this condition already eliminates all negatives, stop here
                    if len(neg_cov) == 0 and len(pos_cov) > 0:
                        best_literal, best_pos, best_neg = (attr, val), pos_cov, neg_cov
                        break

                    # Otherwise, check FOIL gain
                    gain = foil_gain(len(pos_cov), len(neg_cov), len(covered_pos), len(covered_neg))
                    if gain > best_gain:
                        best_gain, best_literal = gain, (attr, val)
                        best_pos, best_neg = pos_cov, neg_cov

                if best_literal and len(best_neg) == 0:
                    break
```

```
            if not best_literal:
                break

            attr, val = best_literal
            rule.append((attr, val))
            covered_pos, covered_neg = best_pos, best_neg

            # Stop if rule already pure
            if covered_neg.empty:
                break

        # -------------------------------
        # PRUNING: remove redundant conditions
        # -------------------------------
        pruned = rule.copy()
        for cond in rule:
            test_rule = [c for c in pruned if c != cond]
            tmp = df.copy()
            for (a, v) in test_rule:
                tmp = tmp[tmp[a] == v]
            if not tmp.empty and all(tmp[target] == target_class):
                pruned = test_rule  # drop redundant condition

        if pruned:
            rules.append(pruned)
            # remove covered positives
            tmp = pos.copy()
            for (a, v) in pruned:
                tmp = tmp[tmp[a] == v]
            pos = pos.drop(tmp.index, errors="ignore")
        else:
            break

    return rules
```

```
# -------------------------------
# 5. Apply Example-based & FOIL
# -------------------------------
print("\n=== Example-based Rules ===")
for attr, val, label in example_based_rules(df, target="Play"):
    print(f"IF {attr}={val} THEN Play={label}")

print("\n=== FOIL Rules for YES ===")
for r in foil(df.copy(), target_class="Yes", target="Play"):
    print("IF", " AND ".join([f"{a}={v}" for a,v in r]), "THEN Play=Yes")

print("\n=== FOIL Rules for NO ===")
for r in foil(df.copy(), target_class="No", target="Play"):
    print("IF", " AND ".join([f"{a}={v}" for a,v in r]), "THEN Play=No")
```

Output

```
Dataset Loaded: (14, 5)
    Outlook Temperature Humidity  Windy Play
0     Sunny         Hot     High  False   No
1     Sunny         Hot     High   True   No
2  Overcast         Hot     High  False  Yes
3      Rain        Mild     High  False  Yes
4      Rain        Cool   Normal  False  Yes

=== Example-based Rules ===
IF Outlook=Overcast THEN Play=Yes

=== FOIL Rules for YES ===
IF Outlook=Overcast THEN Play=Yes
IF Humidity=Normal AND Outlook=Sunny THEN Play=Yes
IF Outlook=Rain AND Windy=False THEN Play=Yes

=== FOIL Rules for NO ===
IF Humidity=High AND Outlook=Sunny THEN Play=No
IF Outlook=Rain AND Windy=True THEN Play=No
```

3. Read a supervised dataset and use bagging and boosting technique to classify the dataset. Indicate the performance of the model.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load EnjoySport dataset
def load_data(file_path):
    df = pd.read_csv(file_path)
    return df

# Encode all categorical features
def encode_categorical(df):
    le = LabelEncoder()
    for col in df.columns:
        df[col] = le.fit_transform(df[col])
    return df
def main():
    file_path = "C:\\Users\\sheet\\OneDrive\\Documents\\ICEAS College\\2025 Odd Sem\\ML2 Lab\\Pgm3\\Weather_extended.csv"
    df = load_data(file_path)
    df = encode_categorical(df)

    X = df.drop("EnjoySport", axis=1)
    y = df["EnjoySport"]

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    # Bagging
    bag_model = BaggingClassifier(
        base_estimator=DecisionTreeClassifier(),
        n_estimators=10,
        random_state=42
    )
    bag_model.fit(X_train, y_train)
    y_pred_bag = bag_model.predict(X_test)

    print("=== Bagging Results ===")
    print("Accuracy:", accuracy_score(y_test, y_pred_bag))
    print("Classification Report:\n", classification_report(y_test, y_pred_bag))

    # Boosting (AdaBoost)
    boost_model = AdaBoostClassifier(
        base_estimator=DecisionTreeClassifier(max_depth=1),
        n_estimators=10,
        random_state=42
    )
```

```
    boost_model.fit(X_train, y_train)
    y_pred_boost = boost_model.predict(X_test)

    print("\n=== Boosting Results ===")
    print("Accuracy:", accuracy_score(y_test, y_pred_boost))
    print("Classification Report:\n", classification_report(y_test, y_pred_boost))

if __name__ == "__main__":
    main()
```

OUTPUT

```
=== Bagging Results ===
Accuracy: 0.5
Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00         2
           1       0.50      1.00      0.67         2

    accuracy                           0.50         4
   macro avg       0.25      0.50      0.33         4
weighted avg       0.25      0.50      0.33         4

=== Boosting Results ===
Accuracy: 0.5
Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00         2
           1       0.50      1.00      0.67         2

    accuracy                           0.50         4
   macro avg       0.25      0.50      0.33         4
weighted avg       0.25      0.50      0.33         4
```

4. Read an unsupervised dataset and group the dataset based on similarity based on k-means clustering .

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# === Step 1: Read dataset from a folder ===
file_path = 'C:\\Users\\sheet\\OneDrive\\Documents\\ICEAS College\\2025 Odd Sem\\ML2 Lab\\Pgm4\\iris.csv'  # Change this to your file path
df = pd.read_csv(file_path)

# === Step 2: Remove non-numeric columns (if any) ===
df_numeric = df.select_dtypes(include=[np.number])

# === Step 3: Standardize the features ===
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_numeric)

# === Step 4: Use Elbow Method to determine optimal k ===
inertia = []
k_range = range(1, 11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42,n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

# Plot the elbow curve
plt.plot(k_range, inertia, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.grid(True)
plt.show()

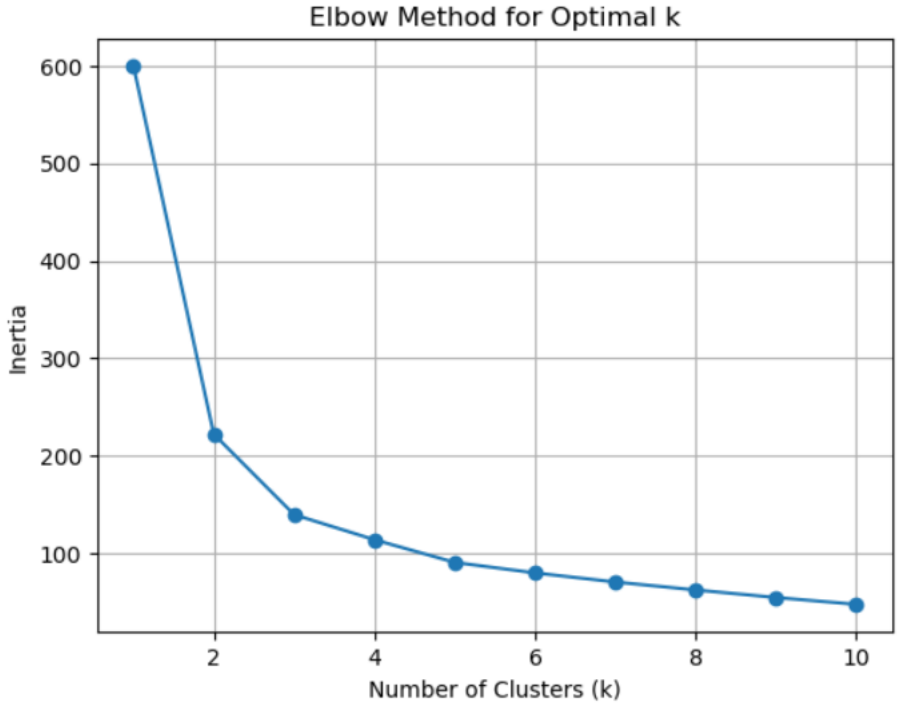
# === Step 5: Apply KMeans with optimal k ===
k_optimal = 3  # Change based on Elbow plot
kmeans = KMeans(n_clusters=k_optimal, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

# Add cluster labels to the original DataFrame
df['Cluster'] = clusters

# Show first few rows with cluster assignments
print("Clustered Data Sample:")
print(df.head())

# Optional: Visualize using first two numeric features
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=clusters, cmap='viridis', s=50)
plt.title("K-Means Clustering Result")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()
```

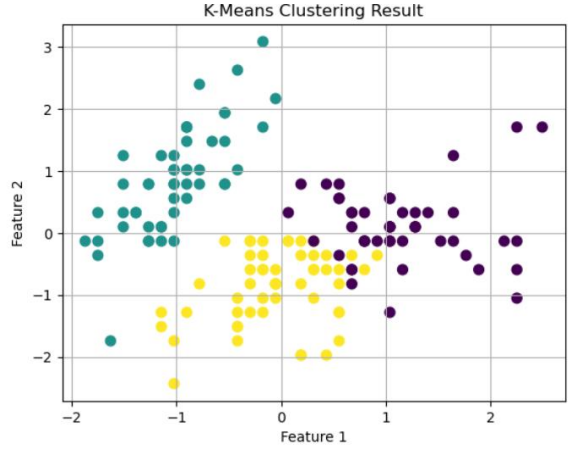
OUTPUT



Clustered Data Sample:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	species	Cluster
0	setosa	1
1	setosa	1
2	setosa	1
3	setosa	1
4	setosa	1



5. Read a dataset and perform unsupervised learning using SOM algorithm.
Install minisom module for SOM implementation

```
!pip install minisom
import pandas as pd
import numpy as np
from minisom import MiniSom
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt

# === Step 1: Load the dataset from CSV ===
file_path = 'C:\\Users\\sheet\\OneDrive\\Documents\\ICEAS College\\2025 Odd Sem\\ML2 Lab\\Pgm4\\iris.csv'  # Change this to your file path
df = pd.read_csv(file_path)

# === Step 2: Select only numeric columns ===
X = df.select_dtypes(include=[np.number]).values

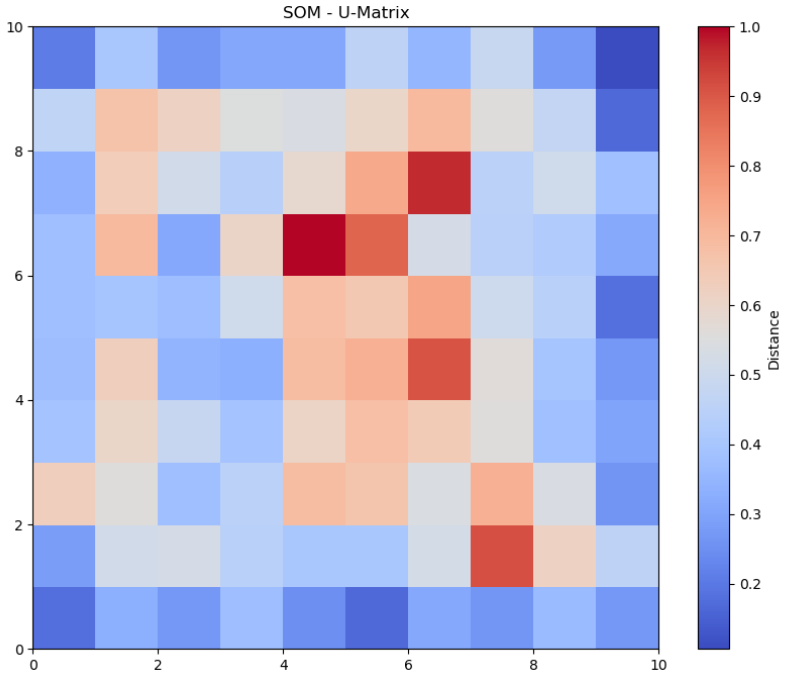
# === Step 3: Normalize data ===
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

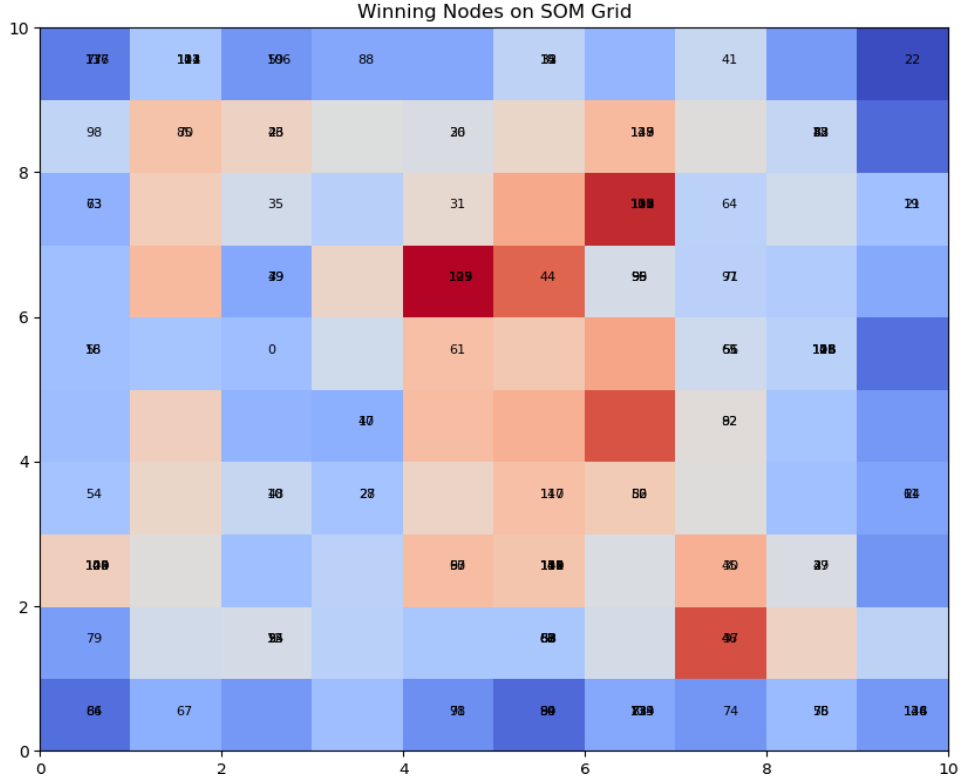
# === Step 4: Initialize and train the SOM ===
som_grid_x, som_grid_y = 10, 10  # SOM grid size
som = MiniSom(x=som_grid_x, y=som_grid_y, input_len=X_scaled.shape[1], sigma=1.0, learning_rate=0.5)
som.random_weights_init(X_scaled)
som.train(X_scaled, num_iteration=100)
```

```
# === Step 5: Visualize SOM (U-Matrix) ===
plt.figure(figsize=(10, 8))
plt.pcolor(som.distance_map().T, cmap='coolwarm')  # U-Matrix (distance map)
plt.colorbar(label='Distance')
plt.title("SOM - U-Matrix")
plt.show()

# === Optional: Mark the winning node for each data point ===
plt.figure(figsize=(10, 8))
for i, x in enumerate(X_scaled):
    w = som.winner(x)
    plt.text(w[0] + 0.5, w[1] + 0.5, str(i), fontsize=8, color='black')
plt.pcolor(som.distance_map().T, cmap='coolwarm')
plt.title("Winning Nodes on SOM Grid")
plt.show()
```

OUTPUT





6. Write a function to generate **uniform random numbers** in the interval [0, 1]. Use this function to generate 10 random samples and evaluate f(x) for each sample. What are the sampled function values? Using the samples generated in the previous step, estimate the integral I using the **Monte Carlo method**.

```
import random

# Step 1: Function to generate uniform random numbers in [0, 1]
def generate_uniform_samples(n):
    return [random.uniform(0, 1) for _ in range(n)]

# Step 2: Define the function f(x)
def f(x):
    return x ** 2

# Step 3: Generate 10 samples
samples = generate_uniform_samples(10)

# Step 4: Evaluate f(x) for each sample and print results
print("Sampled x values and corresponding f(x):")
function_values = []
for i, x in enumerate(samples, 1):
    fx = f(x)
    function_values.append(fx)
    print(f"Sample {i}: x = {x:.4f}, f(x) = {fx:.4f}")

# Step 5: Estimate the integral using Monte Carlo method
estimated_integral = sum(function_values) / len(function_values)
print(f"\nEstimated value of the integral I ≈ {estimated_integral:.4f}")
```

OUTPUT

```
Sampled x values and corresponding f(x):
Sample 1: x = 0.2915, f(x) = 0.0850
Sample 2: x = 0.3946, f(x) = 0.1557
Sample 3: x = 0.3533, f(x) = 0.1248
Sample 4: x = 0.8806, f(x) = 0.7755
Sample 5: x = 0.2727, f(x) = 0.0744
Sample 6: x = 0.7601, f(x) = 0.5777
Sample 7: x = 0.9214, f(x) = 0.8490
Sample 8: x = 0.5436, f(x) = 0.2955
Sample 9: x = 0.2646, f(x) = 0.0700
Sample 10: x = 0.8796, f(x) = 0.7737

Estimated value of the integral I ≈ 0.3781
```

7. Read a dataset and indicate the likelihood of an event occurring using Bayesian Networks.

```
!pip install pgmpy pandas

import pandas as pd
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination

# Step 1: Read the dataset
data = pd.read_csv("C:\\Users\\sheet\\OneDrive\\Documents\\ICEAS College\\2025 Odd Sem\\ML2 Lab\\Pgm 7\\Weather.csv")

# Step 2: Define the structure of the Bayesian Network
# (edges represent dependencies between variables)
model = DiscreteBayesianNetwork([('Rain', 'Traffic'),
                                 ('Traffic', 'Late'),
                                 ('Rain', 'Late')])

# Step 3: Train the model using the dataset
model.fit(data, estimator=MaximumLikelihoodEstimator)

# Step 4: Create an inference object
inference = VariableElimination(model)

# Step 5: Query the likelihood of being Late given Rain = Yes
prob = inference.query(variables=['Late'], evidence={'Rain': 'Yes'})

print("\nProbability of being Late given that it is Raining:\n")
print(prob)
```

OUTPUT

```
Probability of being Late given that it is Raining:

+----------+-------------+
| Late     |   phi(Late) |
+==========+=============+
| Late(No) |      0.4000 |
+----------+-------------+
| Late(Yes)|      0.6000 |
+----------+-------------+
```

8. Refer to the dataset in question 7 and indicate inferences based on the sequence of steps

```
# Install pgmpy before running:
# pip install pgmpy pandas

import pandas as pd
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination

# Step 1: Create or load the dataset
data = pd.read_csv("C:\\Users\\sheet\\OneDrive\\Documents\\ICEAS College\\2025 Odd Sem\\ML2 Lab\\Pgm 7\\Weather.csv")

# Step 2: Define the Bayesian Network structure
# Rain affects both Traffic and Late; Traffic also affects Late
model = DiscreteBayesianNetwork([('Rain', 'Traffic'),
                                 ('Traffic', 'Late'),
                                 ('Rain', 'Late')])

# Step 3: Train the network using Maximum Likelihood Estimation
model.fit(data, estimator=MaximumLikelihoodEstimator)

# Step 4: Create inference object
inference = VariableElimination(model)

# Step 5: Inference 1 — Probability of being Late given Rain = Yes
print("1 P(Late | Rain=Yes):")
q1 = inference.query(variables=['Late'], evidence={'Rain': 'Yes'})
print(q1, "\n")

# Step 6: Inference 2 — Probability of Rain given Late = Yes
print("2  P(Rain | Late=Yes):")
q2 = inference.query(variables=['Rain'], evidence={'Late': 'Yes'})
print(q2, "\n")

# Step 7: Inference 3 — Probability of being Late given Traffic = Yes and Rain = No
print("3 P(Late | Traffic=Yes, Rain=No):")
q3 = inference.query(variables=['Late'], evidence={'Traffic': 'Yes', 'Rain': 'No'})
print(q3)
```

OUTPUT

```
1 P(Late | Rain=Yes):
+-----------+-------------+
| Late      |   phi(Late) |
+===========+=============+
| Late(No)  |      0.4000 |
+-----------+-------------+
| Late(Yes) |      0.6000 |
+-----------+-------------+

2  P(Rain | Late=Yes):
+-----------+-------------+
| Rain      |   phi(Rain) |
+===========+=============+
| Rain(No)  |      0.0000 |
+-----------+-------------+
| Rain(Yes) |      1.0000 |
+-----------+-------------+

3 P(Late | Traffic=Yes, Rain=
+-----------+-------------+
| Late      |   phi(Late) |
+===========+=============+
| Late(No)  |      1.0000 |
+-----------+-------------+
| Late(Yes) |      0.0000 |
+-----------+-------------+
```